

々 To create something from a file

kubectl create -f <file path> [it'll create if resource doesn't exist]

kubectl apply -f <file path> [create if doesn't exist; update if exists]

々 To edit any resource live

kubectl edit <resource type> <resource name>

kubectl edit pod my-pod

々 To see all the details of a resource along with events

kubectl describe <resource type> <resource name>

kubectl describe pod my-pod

々 To see *live resource configuration* in *yaml* format

kubectl get <resource type> <resource name> -o yaml

kubectl get pod my-pod -o yaml

々 Too see all the things of the available resources

kubectl get <resource type> <resource name> -o wide

々 To get a *yaml* configuration file of any resource to start with,

--dry-run=client -o yaml append this in the imperative commands.

kubectl create deployment my-dep --image=nginx --dry-run=client -o yaml

々 Pod

♣ To create a pod ("v1")

kubectl run <pod name> --image=<image name>

kubectl run my-pod --image=nginx

♣ Whatever ports you write inside containers (inside pods) are not exposed, that is just used for user's docs or to search pods a/c to ports. The traffic reaches to a specific port of a container with Service only.

♣ If a pod contains multiple containers,
each container: *same IP, different ports* [because IP is assigned to Pod]

々 ReplicaSet

♣ ReplicaSet ("apps/v1") and ReplicationController ("v1") cannot be created with imperative commands.

♣ ReplicaSet contains [*replicaset.spec.selector*](#) field, which ReplicationController doesn't have.

♣ Selector contains [*matchLabels*](#) or [*matchExpressions*](#).

- ⌘ *template* (pod) should contain same or more labels present in *selector*.
Selector's labels work with AND rule, not OR. All the labels must be present in the pods to be selected.

- ⌘ To scale replica set:

kubectl scale --replicas=<replica count> replicaset <replicaset name>

kubectl scale --replicas=4 replicaset my-rs

✧ Deployment

- ⌘ Deployment can be created with Imperative way because Kubernetes auto-fill the required complex fields like template, labels, selectors.

kubectl create deployment <deployment name> --image=<image name> --replicas=<replica count>

- ⌘ It provide *pull*, *upgrade*, *rolling update (or) recreate*, *rollback (undo)*, *pause (rollout pause)*, *resume (rollout resume)* over ReplicaSet.

- ⌘ It create new versions, and *scales down* old ReplicaSet gradually and *scales up* new ReplicaSet gradually.

- ⌘ [*maxUnavailable*](#) (maximum allowed unavailable pods. Percentage or count), [*maxSurge*](#) (maximum allowed available pods. Percentage or count)

[scale up of new pods & scale down of old pods happens w.r.t.

maxUnavailable & *maxSurge*]

- ⌘ To see *rollout status*,

kubectl rollout status deployment <deployment name>

To see *rollout history*,

kubectl rollout history deployment <deployment name>

[for this --record has to be provided; but its deprecated]

To *undo* the deployment

kubectl rollout undo deployment <deployment name>

- ⌘ Deployment = ReplicaSet + [rollout, rollback, versioning(revisions), rollout strategy (Recreate, RollingUpdate), Pause, Resume]

✧ Deployment Strategy

- ⌘ [*RollingUpdate*](#): gradually remove old pods and create new pods.

- ⌘ [*Recreate*](#): remove all old pods and create new pods.

✧ Service

- ⌘ Required to communicate with the pods.
[Pod's IP can be used; but shouldn't because Pods might get re-created]
- ⌘ External to Pod: **NodePort**
Pod to Pod: **ClusterIP** (NodePort can also be used; but not preferred)
Load Balancing: **LoadBalancer** [works in cloud based clusters]
- ⌘ type: NodePort
ports: [{ nodePort: x, port: y, targetPort: z }, { .. }, ..]
port means service's port
in here, whatever port you assign for *service*, it doesn't matter because it is there only to connect nodePort to targetPort (pod's port).
- ⌘ type: ClusterIP
ports: [{ port: x, targetPort: y }, { .. }, ..]
- ⌘ For inter-pod communication
<service name>:<service port>
service -> name, selector; pod: labels; <-- these are important
- ⌘ *selector* doesn't contain *matchLabels*, *matchExpressions*.
Direct *key:value* pairs (labels) to select the target pods.
- ⌘ Service's redirection to Pod is cluster level. So service's and pod's host nodes might be different.
- 🔗 **kubeadm** tool can be used to create a cluster and configure multiple nodes (control plane & worker nodes). it generates the required certificates and all.
<https://github.com/kodekloudhub/certified-kubernetes-administrator-course/tree/master/kubeadm-clusters/generic>
- 🔗 **cgroup** (Control Group)
 - ⌘ Used to enforce limits by Kernel.
 - ⌘ **mount | grep cgroup** : to see the mounted path (/sys/fs/cgroup)
cgroup2 for modern systems; cgroup for older systems;
 - ⌘ Files & Directories inside *cgroup* are handled by ?
 - ⌘ all files : *Kernel*
 - ⌘ directories of format *cgroup.**, *cpu.**, *memory.**, *io.** : *Kernel*
 - ⌘ directories of format *.slice*, *.scope*, *.service*, *.mount* : **systemd**
 - ⌘ other directories (*docker*, *kubepods* ..etc): respective owners

- ♣ *system.slice* contains *containerd.service*, it means container runtime (containerd) is managed by *systemd* itself.

In case of separately handled, both will not get to know each other's usage and request for more resources; at the end kernel will take action and stop some resources which might be important.

- ♣ Owners writes into the respective files, Kernel enforces limits.

✧ Internals of *kubectl apply*

- ♣ 3 configs are there:

config file (static yaml configuration file used to create a resource)

live config file (configuration file of the running resource; *kubectl get <resource type> <resource name> -o yaml*)

last applied config (latest config applied in *declarative* way) [its in JSON format]

- ♣ Running resource updated with:

- ♣ imperative command (or) *kubectl edit* command: live config is updated, last applied config will remain same
- ♣ Declarative way (*kubectl apply*) : live applied config will be updated, config file will be updated of-course.

- ♣ Configs updation:

If a field is there in *live config* (ex: an extra level was added using *kubectl edit* command) but not in both *config file* and *last applied config* then field will remain as it is.

If a field is there in either *config file* or *last applied config* or both then that field will be updated (doesn't matter if that field is there in *live config* or not)

Field in object configuration file	Field in live object configuration	Field in last-applied-configuration	Action
Yes	Yes	-	Set live to configuration file value.
Yes	No	-	Set live to local configuration.
No	-	Yes	Clear from live configuration.
No	-	No	Do nothing. Keep live value.

✧ namespace

- ♣ Namespace defines a logical boundary where resources can be present.

- Same named resources cannot stay inside a single namespace; but can be present in different namespaces.
- Ex: dev, prod, ..etc namespaces can be created.
- To query/update resources in a specific namespace, use
`-n <namespace name>` (or) `--namespace <namespace name>`
`kubectl get pods -n dev-namespace`
- In the yaml, `metadata.namespace` is the required field to set the namespace.
- Clustered configured with `kubeadm`, contains `kube-system` namespace which contains control plane resources.
- To access a service of another namespace
`<service name>.<target namespace name>.svc.cluster.local`
`db-service.dev-namespace.svc.cluster.local`

ResourceQuota

- Used to limit resources for **namespace** level.

```
spec:
  hard:
    pods: "10"
    requests.cpu: "4"
    requests.memory: 5Gi
    limits.cpu: "10"
    limits.memory: 10Gi
```

Scheduling

- `Scheduler` updates `pod.spec.nodeName` field of Pod.
 Then request API server to bind Pod to the specific node.
 Binding request contains payload, where `kind: bind`

```
apiVersion: v1
kind: Binding
metadata:
  name: test-binding
target:
  apiVersion: v1
  kind: Node
  name: node01
```

[not a Kubernetes object; just payload of request]

- `curl <request uri> --request POST --data '{"apiVersion": "v1", ...}'`
 (bind config in JSON format)

- ⌘ In case of manual scheduling, mention [*pod.spec.nodeName*](#) field and it'll create pod in required node.
- 々 To query resources with specific labels,
 - selector key1=val1,key2=val2** [append this]
 - `kubectl get all --selector app=nginx,key2=val2`
- 々 [*metadata.annotations*](#) used for giving extra info like build version info, adding tool details ..etc without impacting anything.
- 々 Taints & Tolerations
 - ⌘ Taint is applied on Node; Toleration is applied on Pod.
 - ⌘ Taint prevent pod to be scheduled to the tainted node;
 - If Pod is tolerated against that specific taint then only it can be scheduled to that tainted node.
 - ⌘ To taint node (*imperative method only; no declarative method*)
 - kubectl taint node <node name> <key>=<value>:<taint effect>**
 - `kubectl taint node node01 env=prod:NoSchedule`
 - ⌘ To remove taint, just append **-** at the end.
 - kubectl taint node <node name> <key>=<value>:<taint effect>-**
 - `kubectl taint node node01 env=prod:NoSchedule-` (most specific)
 - `kubectl taint node node01 env=prod-` (less specific)
 - `kubectl taint node node01 env-` (more less specific)
 - ⌘ One node can have more than one same key-value pair taints; but *taint effect* should be different.
 - `env=prod:NoSchedule`
 - `env=prod:NoExecute`
 - `env=prod:PreferNoSchedule`
 - ⌘ To tolerate the pod (*declarative method only; no imperative method*)
 - update the field [*pod.spec.tolerations*](#) [it contains a list]

```
spec:
  tolerations:
  - key: "env"
    operator: "Equal"
    value: "prod"
    effect: "NoSchedule"
```

 - operators: *Equal, Exists, Gt, Lt*
 - double quote is mandatory here.
 - ⌘ Taint effects:

- [NoSchedule](#): all current pods will present; non-tolerated pod will not be scheduled.
- [PreferNoSchedule](#): all current pods will present; non-tolerated pod might be scheduled in case of no other option.
- [NoExecute](#): all current non-tolerated pod will be removed; non-tolerated pod will not be scheduled.

✧ nodeSelector

- [pod.spec.nodeSelector](#) contains the labels (key:value pairs)
all the labels matches with the node => node is scheduled to this pod.

```
spec:
  nodeSelector:
    env: prod
```

✧ nodeAffinity

- [pod.spec.affinity.nodeAffinity](#) has to be updated.
nodeAffinity support complex queries like DoesNotExist, Exists, Gt, In, Lt, NotIn
 which was not possible in [nodeSelector](#)
- nodeAffinity is of 2 types:
 - [requiredDuringSchedulingIgnoredDuringExecution](#) [HARD rule]
 - [preferredDuringSchedulingIgnoredDuringExecution](#) [SOT rule (Scored basis)]

```
nodeAffinity
├─ preferredDuringSchedulingIgnoredDuringExecution []
│  └─ preference
│     └─ matchExpressions []
│        └─ matchFields []
│           └─ weight
└─ requiredDuringSchedulingIgnoredDuringExecution
   └─ nodeSelectorTerms []
      └─ matchExpressions []
         └─ matchFields []
```

- [~~its not match~~**Labels**, its **matchFields** (not limited to labels, can query any field like [metadata.name](#))]
- [matchExpressions](#): query1 **AND** query2 **AND** ...
- [matchFields](#): query1 **AND** query2 **AND** ...

```

matchExpressions:
- key: env
  operator: In
  values:
  - prod
- key: disk
  operator: In
  values:
  - ssd

matchFields:
- key: metadata.name
  operator: In
  values:
  - node01
  - node02
- key: metadata.resourceVersion
  operator: In
  values:
  - "353007"
  - "353008"

```

⌘ requiredDuringSchedulingIgnoredDuringExecution

```

requiredDuringSchedulingIgnoredDuringExecution
├─ nodeSelectorTerms [] (OR)
│   └─ Term 1
│       ├── matchExpressions [] (AND)
│       └─ matchFields [] (AND)
│           ⇒ Term 1 = (matchExpressions AND matchFields)
│
│   └─ Term 2
│       ├── matchExpressions [] (AND)
│       └─ matchFields [] (AND)
│           ⇒ Term 2 = (matchExpressions AND matchFields)

```

- ⌘ nodeSelectorTerms [] = term1 OR term2 OR ...
each term = (*matchExpressions* AND *matchFields*)
- ⌘ Node should satisfy any one *term* (means all queries of *matchExpressions* AND *matchFields* of that *term*)

⌘ preferredDuringSchedulingIgnoredDuringExecution

```

preferredDuringSchedulingIgnoredDuringExecution [] (scored list)
├─ Item 1
│   ├── preference
│   │   ├── matchExpressions [] (AND)
│   │   └─ matchFields [] (AND)
│   │       ⇒ match = (matchExpressions AND matchFields)
│   └─ weight
│
├─ Item 2
│   ├── preference
│   │   ├── matchExpressions [] (AND)
│   │   └─ matchFields [] (AND)
│   │       ⇒ match = (matchExpressions AND matchFields)
│   └─ weight

```



```
[
  { preference: (matchExpressions AND matchFields), weight },
  { ... },
  ...
]
```

Each item's preference:

$\text{weight} = \text{'provided weight' if match else '0' (zero)}$

$\text{total score of node} = \text{sum of each item's weight}$

requiredDur....Execution: at least one should match

preferredDur...Execution: more matches, more score

Resource Requirements

Update field: `pod.spec.containers[*].resources`

`requests` -> required resources, else node will not be scheduled

[scheduling phase].

`limits` -> maximum utilization of resources in the node [running phase]

```
resources:
  requests:
    cpu: "500m"
    memory: "256Mi"
  limits:
    cpu: "1"
    memory: "512Mi"
```

cpu smallest unit: **1m** (1 mili CPU), **1** means one full CPU.

memory smallest unit: **1** (1 byte), use **Gi, Mi, Ki** (don't use G, M, K)

CPU maximum limit reached: throttled (slow down)

Memory maximum reached: container is killed

Best scenario: Request, No Limit

LimitRange

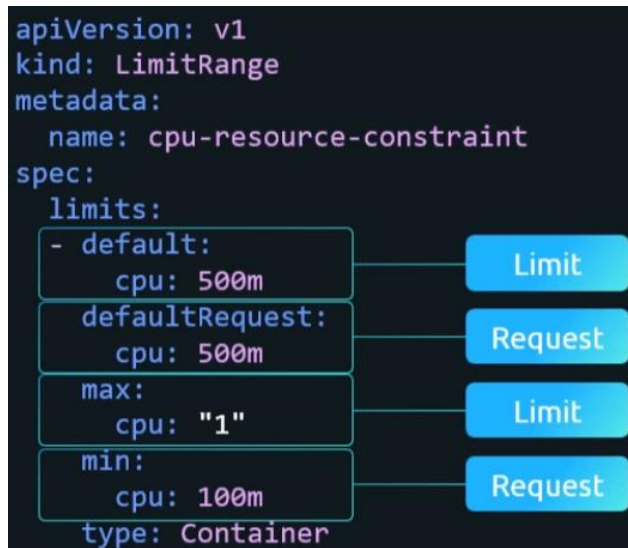
Inject default *request* and *limit* if not configured inside pods.

Multiple LimitRange can be configured in a namespace, but they should not conflict.

Ex: one says `max cpu: 500m`, another says `min cpu: 600m`

in this case, pod will not be created because of conflicting values.

- ☞ In real life: one LimitRange per namespace is maintained.



limits.default : limit

limits.defaultRequest: request

limits.max: request or limit should not more than it.

limits.min: request or limit should not less than it.

- ♪ ResourceQuota is for namespace level.

LimitRange is for Pod level.

- ♪ DaemonSet

same as ReplicaSet.

kind: DaemonSet

- ☞ It make sure one copy of the Pod runs on each node.

- ♪ Static Pods

- ☞ Kubelet on each node watches a directory, if Pod's yaml config file is present then it creates a pod.

It doesn't involve API Server.

- ☞ **systemctl cat kubelet** [see the *kubelet.service* file]

get the *config.yaml* file path and open that file.

One key will be there staticPodPath, that is the path to keep the Pod's yaml configs.

- ☞ Kube API Server knows about all the static pods, but you cannot update using **kubectrl edit** command.

The *yaml* config file needs to be changed to update the static pod.

- ♪ Pod creation internals

- ⌘ `kubectl apply -f my-pod.yaml` [pod creation request triggered]
- ⌘ API server validates, & creates Pod object. Update etcd about *PodSpec*.
- ⌘ Scheduler sees an unscheduled pod, assign a node.
- ⌘ *Kubelet* has a watch connection with API Server. Changes are streamed to kubelet from API server through that (its not webhook; kind of streaming type)

```
GET /api/v1/namespaces/test/pods?watch=1&sendInitialEvents=true
---
200 OK
Transfer-Encoding: chunked
Content-Type: application/json

{
  "type": "ADDED",
  "object": {"kind": "Pod", "apiVersion": "v1", "metadata": {"name": "my-pod", "namespace": "test"}, "spec": {"containers": [{"name": "my-container", "image": "nginx"}]}},
{
  "type": "ADDED",
  "object": {"kind": "Pod", "apiVersion": "v1", "metadata": {"name": "my-pod", "namespace": "test"}, "spec": {"containers": [{"name": "my-container", "image": "nginx"}]}},
{
  "type": "BOOKMARK",
  "object": {"kind": "Pod", "apiVersion": "v1", "metadata": {"name": "my-pod", "namespace": "test"}, "spec": {"containers": [{"name": "my-container", "image": "nginx"}]}}
```

- ⌘ Kubelet sees *PodSpec*, pull it, command Container Runtime.
- ⌘ Container Runtime pull the required image, create the containers, assigns Container IDs.
- ⌘ Kubelet maps *Pod ID* -> *Container IDs* .
monitors containers (running/crashed/restarted)
- ⌘ Kubelet updates API Server about Pod's status (send details about containers like container IDs, pod status, ..etc)
- ⌘ API Server stores details in etcd.
[each few seconds, Kubelet updates API Server about the pod status, container details ..etc]
- ⌘ Container runtime knows about container only (container ID and all)
Kubelet knows about *PodSpec* (map it to container IDs received from container runtime)
API Server know keeps info that got from Kubelet only.
- ⌘ Each time we execute *kubectl edit* or *kubectl apply -f*, API Server sends *PodSpec* and the process continues.

- ⌘ In case of static pods, Kubelet sends a few extra fields
`metadata.annotations."kubernetes.io/config.source": file`
`metadata.annotations."kubernetes.io/config.mirror": <hash>`

🔗 Resources communication



- ⌘ All resources talk to API Server, API server talks to *etcd*.
All the communications happen via HTTP calls.
- ⌘ So, even if the resources are running in the node directly
(or)
the resources are running inside pods, it doesn't matter.
Node to Pod will have some port mapping (in case of *kubeadm*, port mappings are same i.e. 6443:6443 ..etc)
- ⌘ The pods that are running resources, must be static. Because API Server shouldn't create or delete these.
- ⌘ **Kubelet** doesn't run inside any Pod. It is a **systemd** service.

🔗 Multiple Schedulers

- ⌘ 3 files are involved:
`kube-scheduler.yaml` [kind: Pod , static pod that runs scheduler]
`scheduler.conf` [config file containing *cluster*, *user*, *context* details]
`scheduler-config.yaml` [kind: KubeSchedulerConfiguration; optional]
- ⌘ Either you mention the things in `pod.spec.containers[0].command` field without mentioning `scheduler-config.yaml` file [minimal configs like **--leader-elect=true**]
(or)
mention the `scheduler-config.yaml` file for proper configuration.
--config=/etc/kubernetes/scheduler-config.yaml [you can give any name]

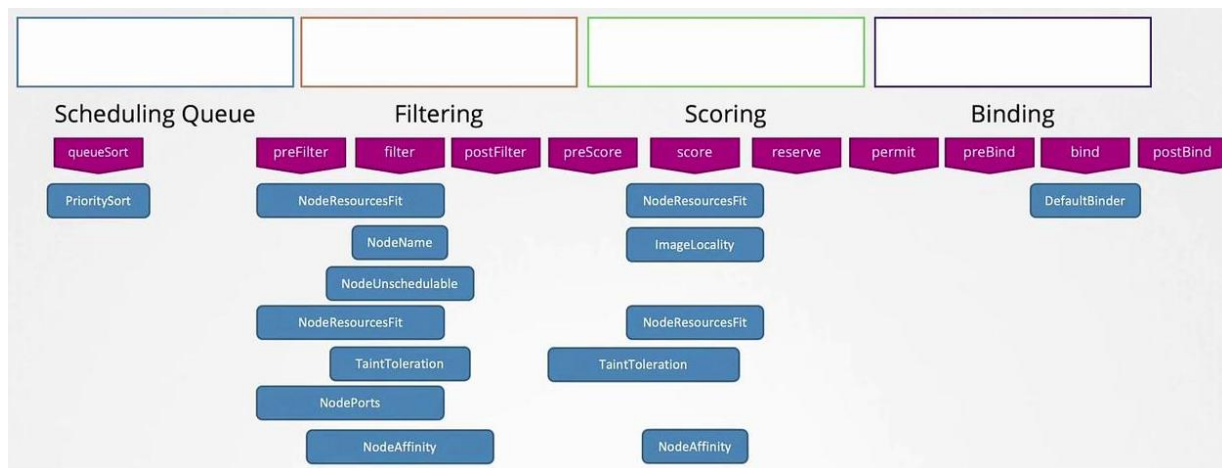
```
spec:
  containers:
  - name: kube-scheduler
    image: k8s.gcr.io/kube-scheduler-amd64:v1.11.3
    command:
    - kube-scheduler
    - --address=127.0.0.1
    - --kubeconfig=/etc/kubernetes/scheduler.conf
    - --config=/etc/kubernetes/my-scheduler-config.yaml
```

- ♣ To use this scheduler to schedule your pod,
pod.spec.schedulerName field has to be updated.

```
schedulerName: my-custom-scheduler [ pod.spec.schedulerName ]
```

🔗 Scheduler Profiles

- ♣ Scheduler is a process (one pod)
- Scheduler profile is a rule.
 - ♣ One scheduler can have multiple scheduler profiles.
- ♣ Scheduling has different stages called **execution point**.
queueSort,
preFilter, filter, postFilter,
preScore, score, reserve,
permit, preBind, bind, postBind
- ♣ At each extension point, modules are bound.



- ♣ Scheduler profiles runs on different threads.
- In each scheduler profile, you can decide which plugins has to be

enabled or disabled.

```
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: my-scheduler-2
  plugins:
    score:
      disabled:
      - name: TaintToleration
      enabled:
      - name: MyCustomPluginA
      - name: MyCustomPluginB
```

- ⌘ KubeSchedulerConfiguration is not a Kubernetes object.

That's why this file's path is given in kube-scheduler.yaml

⌘ Leader Selection

- ⌘ [kubeschedulerconfiguration.leaderElect](#): true (or) false
- ⌘ Enable: one scheduler will be active at a time
Disable: multiple schedulers will be active at a time.
- ⌘ In case of race condition, Kubernetes handles it.
Only one scheduling can happen. No re-scheduling.
- ⌘ Leader election should intelligently be chosen.
scheduler -> s1: p1 p2 (profiles)
scheduler -> s2: p3 p4
if you have made *leader election* true, the pod has *schedulerName*: p3
and if s1 is chosen as leader, it doesn't have p3, pod will be in pending state forever.

⌘ Types of *Authorizations mechanisms*:

- ⌘ Node Authorizer [for kubelet]
- ⌘ ABAC [Attribute Based Access Control]
- ⌘ RBAC [Role Based Access Control]
- ⌘ Webhook Mode

⌘ Authentication in Kubernetes

- ⌘ Kubernetes itself doesn't provide authentication by itself because it doesn't have database to store credentials.

It depends on external providers like *static file*, *certificate authorities*, *third party identity provider* like *LDAP*.

- ⌘ All authentication & authorization is done at API Server level, because it is the single point of contact to access the cluster.

々 Static file authentication

- ⌘ csv file is maintained to write the password/token, username, user id, group id (optional)

- ⌘ `password123,user1,u0001,group1` (password)

`KpjCvBI7rCFAHYPKByTlzRb7guLcUc4B,user10,u0010,group1` (token)

[group is optional]

- ⌘ Now mention this file's path in the *kube-apiserver* running service (either running as a process or running inside a pod)

```
ExecStart=/usr/local/bin/kube-apiserver \  
--authorization-mode=Node,RBAC \  
--advertise-address=... \  
--basic-auth-file=user-details.csv
```

```
command:  
- kube-apiserver  
- --authorization-mode=Node,RBAC  
- --advertise-address=172.17.0.107  
- --basic-auth-file=/path/to/user-details.csv
```

- ⌘ Curl command usage:

`curl -v -k https://master-node-ip:6443`

for password: `-u "username:password"`

for token: `--header "Authorization: Bearer 45urtgfhy9thgi5494"`

々 Certificate generation process:

- ⌘ One entity (or user) will create a key (signature).
it'll create a certificate about its identity and sign that with its signature [self-signed]. <---- CA
- ⌘ It'll be told that, that guy is the source of truth.
If he signs in any certificate, then that certificate will valid (proof).
- ⌘ Now, any other user will create a *certificate signing request* and the certificate authority (CA) will sign this with its *key* (signature).
now, that user will get a certificate which will be proof of it to be valid.

- ♣ In Kubernetes,
one user will generate *key*, then a *csr*.
csr will be self-signed, then *crt* will be generated.
At the end: **ca.key**, **ca.crt** (signature, CA's identity proof)
 - ♣ For all the other users, they will create *key*, *csr*.
crt will be generated with sign of CA.
- ✧ Client & Server components in Kubernetes
 - ♣ ONLY CLIENT
kubectl, *kube-scheduler*, *kube-controller-manager*, *kube-proxy*
[all talks to API Server]
 - ♣ BOTH SERVER and CLIENT
kube-api-server
[client: *etcd*, *kubelet* ; server: *kubelet*, other components]
kubelet
[client: *api-server* ; server: *api-server*]
etcd
[client: peer-to-peer ; server: *api-server*]
- ✧ OpenSSL commands
 - ♣ **openssl generate -out <key file name>.key 2048**
create a key of size 2048 and store in a file.
openssl genrsa -out server.key 2048
 - ♣ **openssl req -new -key <key file name>.key -subj
"/CN=<name>/O=<group>" -out <csr file name>.csr**
create certificate signing request file.
openssl req -new -key server.key -subj "/CN=etcd" -out server.csr
 - ♣ **openssl x509 -req -in <csr file name>.csr -CA <ca crt file name>.crt
-CAkey <ca key file name>.key -out <crt file name>.crt**
create a certificate signed by CA.
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -out server.crt
 - ♣ **openssl x509 -in <crt file name>.crt -noout -subject**
display subject of the certificate (CN, O ..etc)
 - ♣ **openssl x509 -in <crt file name>.crt -noout -issuer**
display the CA who signed this certificate.

々 Encode and Decode **base64** format

- To convert *key* or *crt* file into *base64* format,

cat <crt file name>.crt | base64

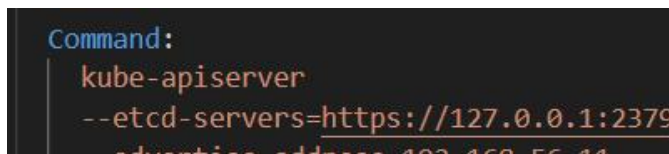
cat <crt file name>.crt | base64

- To convert *base64* to *key* or *crt*

cat <base64 content file name> | base64 -d

々 **etcd peer-to-peer communication**

- All etcd are *etcd servers*. [~~no etcd client; nothing exist like that~~]
- All *etcd servers* contains same data [data replication].
- etcd servers end-points are *static*.
- All the *etcd servers* end-points are provided to *kube-api-server*.



```
Command:
kube-apiserver
--etcd-servers=https://127.0.0.1:2379
advertise-address 192.168.56.11
```

- *kube-api-server* just connect to one *etcd server's end-point*,
etcd cluster handles *leader selection*, *data consistency*, *request routing*.

々 TLS in Kubernetes

- Generate **CA** key and certificate.

Key: **openssl genrsa -out ca.key 2048**

CSR: **openssl req -new -key ca.key -subj "/CN=**KUBERNETES-CA**" -out ca.csr**

CRT: **openssl x509 -req -in ca.csr -signkey ca.key -out ca.crt** [self signed]

- ✦ CN: KUBERNETES-CA is just a *convention*, its never used logically.

Trust is built with **ca.crt** itself.

Keeping this name helps in analyzing logs and all.

- Admin (kubectrl) key and certificate (Client)

Key: **openssl genrsa -out admin.key 2048**

CSR: **openssl req -new -key admin.key -subj "/CN=**kube-admin**/O=**system:masters**" -out admin.csr**

CRT: **openssl x509 -req -in admin.csr -CA ca.crt -CAkey ca.key -out admin.crt** [signed by CA]

- ✦ /O : organization (or) group

- ✦ /CN : common name (or) username

- ✧ **system:masters** group is bound with **cluster-admin** (cluster role).
so, user having certificate containing group *system:masters* will have admin authorization.
- ✧ To find these:
 - search for specific roles or cluster roles
 - search for rolebindings or clusterrolebindings binding those role/clusterrole
- ⌘ Similarly, generate key and certificates for other clients (CN will only be changed).
- ⌘ Server keys and certificates will be created same way.
- ⌘ etcd key and certificate
same command, CN=etcd
 - ✧ **etcd has its own CA** (not same as Kubernetes CA)
it doesn't talk to any other component as a client. Only talks to peer etcd servers.
 - ✧ Its CA has name: **etcd-ca**
 - ✧ *etcd* has 2 certificates: *server.crt*, *peer.crt* [both are signed by etcd CA]
 - ✧ kube-api-server connects to etcd: *server.crt*
etcd connects to peer etcd: *peer.txt*

```
- command:
- etcd
- --cert-file=/etc/kubernetes/pki/etcd/server.crt

- --peer-cert-file=/etc/kubernetes/pki/etcd/peer.crt
- --peer-client-cert-auth=true
- --peer-key-file=/etc/kubernetes/pki/etcd/peer.key
- --peer-trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt

- --trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt
```

- ✧ Kubernetes CA & etcd CA don't trust each other.
They both are completely independent.
- ⌘ kube-api-server key and certificate (Server & Client)
 - ✧ 2 certificates, in both CA are different.
 - ✧ One signed by **Kubernetes's CA**
another signed by **etcd CA**

- ✧ *certificate* is not global. It is used per connection.
- ✧ kube-api-server as server: Kubernetes's CA signed crt
kube-api-server as client: etcd CA signed crt.
- ✧ apiserver-etcd-client.crt : talking as a client to etcd
apiserver-kubelet-client.crt : talking as a client to kubelet
- ✧ kubelet key and certificate (Server & Client)
 - ✧ Server certificates: (api server as client)
client certificate (talking to other peers)
 - ✧ For client certificate, the naming format must be
system:node:<node name> (username)
system:nodes (group)
system:node:node01
both username and group must be in this format. Otherwise not
authorized. (Node Authorizer)
- 々 SSL verification flow
 - ✧ Client sends request
 - ✧ Server asks for certificate
 - ✧ Client sends certificate
 - ✧ Server verifies CA.
Extracts name (CN) & group (O) then applies RBAC.
- 々 **kubeconfig**
 - ✧ Config file contains 3 main things: clusters, contexts, users
 - ✧ **clusters**
it a list. Each index contains 1 cluster's details, i.e. *name* (just an identifier), *cluster's end-point* (cluster's api server's end point [6443 port]), *cluster's CA certificate*.
 - ✧ **users**
its also a list. Each index contains 1 user's details, i.e. *name* (just an identifier), *client key* and *client certificate*.
 - ✧ **contexts**
its also a list. Each index contains a context.
context = user + cluster + default namespace.
each context contains a *name* (just an identifier), *user* (name of the

user), *cluster* (name of the cluster), *namespace* (optional; if not given *default* namespace will be considered by default)

☞ when you execute any command, by default it'll append

--namespace=<provided namespace in context>

however, you can override this by giving

--namespace=<some other namespace>

☞ Context name convention: *<user name>@<cluster name>*

↗ **current-context** : shows the context you are currently in.

↗ *User or cluster can't be switched, need to switch context to get a particular user & cluster pair.*

↗ **kubectl config view**

displays the config file.

kubectl config view --raw

it'll display certificates and keys as well.

↗ **kubectl config use-context <context name>**

to switch context (updates *current-context* field in config file)

↗ **kubectl config set-context <context name> --cluster=<cluster name> --user=<user name> --namespace=<namespace name>**

if context exists: update that; if not: create a new context.

cluster name & user name should be there in config file.

namespace is optional.

↗ Default kube config file path: *~/.kube/config*

to use any other config file append

--kubeconfig <config file path>

kubectl get pods --kubeconfig /path/to/config

々 Use **jsonpath** to extract value of specific key.

kubectl config view --raw -o jsonpath='{users[0].user.client-key-data}'

々 To communicate with api server with directly http REST call

curl https://<controlplane node ip>:6443 --cert /path/to/crt --key /path/to/key --cacert /path/to/ca.crt

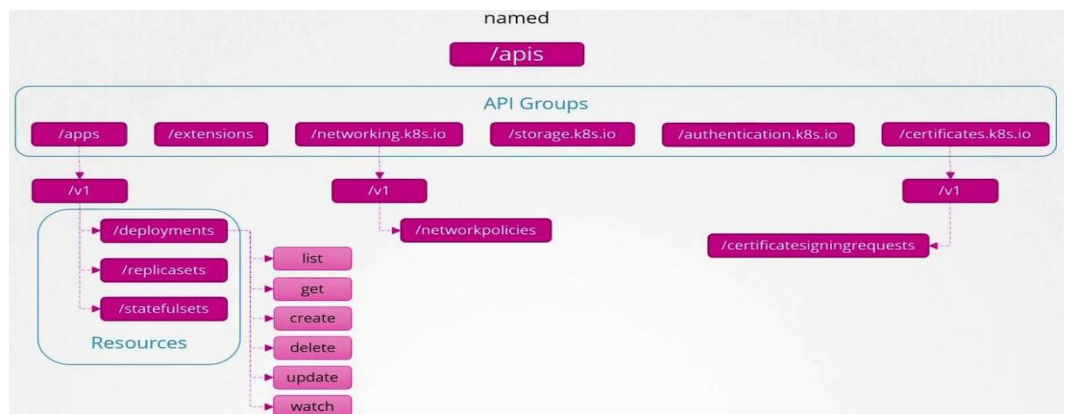
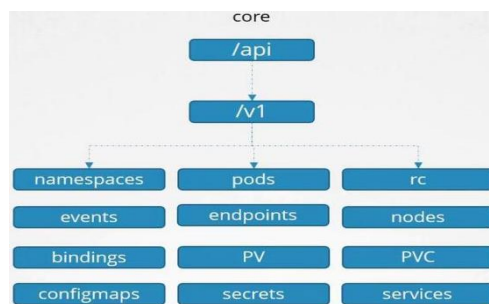
[only in case of certificate authentication, otherwise authorization or username password]

々 GET request to api-server end-point gives all the available REST paths

```
"/apis/",  
"/apis/admissionregistration.k8s.io",  
"/apis/admissionregistration.k8s.io/v1",  
"/apis/apiextensions.k8s.io",  
"/apis/apiextensions.k8s.io/v1"
```

々 Kubernetes Resource API

- ⌘ **/api** and **/apis** contains all the resources.
Other paths like /metrics, /healthz, /version, /logs ..etc are operational, debugging or meta end-points.
- ⌘ **/api** : core Api root path (legacy)
/apis : named Api root path (modern)
- ⌘ 3 things are there to be precised:
resources (pod, events, replicaset ..etc)
resources versions (currently all are v1 only)
resources types (resources are grouped; called API Groups)
- ⌘ **resource** $\subset$ **version** $\subset$ **API groups** ($\subset$: subset)
- ⌘ **/api** is legacy, it doesn't contain api groups.
It contains version directly (**v1**) and resources inside those.
- ⌘ **/apis** is modern, it contains api groups.
Each api group contains version (currently **v1** only).
each version contains resources.



- Each resource has **verbs** (get, list, create, delete ..etc) *actions that can be performed with resources.*
- In *yaml* file, against [apiVersion](#) field:
 - "<version>" : (/api), because api group is not present here
 - "<api group>/<version>" : (/apis), api groups are here
- [apiVersion](#): "v1"
- [apiVersion](#): "apps/v1"
- [official docs](#)

✧ Authorization mechanisms are provided in *kube-apiserver* command (for pod) or *ExecStart* (for process)

```
- command:
  - kube-apiserver
  - --authorization-mode=Node,RBAC
```

- authorization-mode=<mechanism 1>,<mechanism 2>, ..etc**
- It is executed from **left to right** until the client is allowed.
- If till the last authorization mechanism all rejects, then client is unauthorized.

✧ ServiceAccount

- Unlike User account, ServiceAccount is used for services (internal components or external tools like jenkins, prometheus) to communicate with API Server.
- Example: the pods contains *default ServiceAccount* by default.

```
vagrant@controlplane:~/.kube$ kubectl describe pods dev-pod
Name:          dev-pod
Namespace:     default
Priority:       0
Service Account: default
```

[~~inter pod communication doesn't require API server~~]

[multiple reasons for Pod to be needing ServiceAccount to talk to API server]

- ServiceAccount is a Kubernetes resource (core API root path)
- [long lived object]
- For internal components (like pods ..etc)
- token* is created linked to the ServiceAccount by default.

For external tools

token has to be created manually.

✧ token is required to communicate with API server.

✧ **kubectl create token** <service account name>

[token is not a long lived resource]

[token linked to a ServiceAccount has all the authorization provided to ServiceAccount]

✧ a virtual volume is created and mounted to the pod which contains ServiceAccount related things.

```
Mounts:
  /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-rdvhr (ro)
```

virtual volume lifecycle = pod's life cycle [if pod is gone, virtual volume is gone]

to disable this auto mounting of volume:

pod.spec.automountServiceAccountToken: false

✧ **Bearer <token>** can be used in headers to communicate.

々 Node Authorizer (Authorization mechanism 1)

✧ Used by **kubelet**. Its certificate contains:

system:node:<node name> [name (CN)]

system:nodes [group (O)]

✧ This convention cannot be changed. Its fixed for Kubernetes Node Authorizer.

々 **ABAC** [Attribute Based Access Control] (Authorization mechanism 2)

✧ It uses a policy file (JSON file).

contains list of dictionaries (each dictionary: policy)

```
{
  "user": "viewer",
  "apiGroup": "*",
  "resource": "*",
  "namespace": "*",
  "readonly": true
},
```

["*" means all access]

[" in apiGroup doesn't mean all access, it means core api root path]

```
{
  "user": "monitor",
  "nonResourcePath": "/metrics",
  "readonly": true
}
```

⌘ [for non resource paths]

⌘ Policy file path is given in *kube-apiserver* command:

```
--authorization-mode=ABAC
--authorization-policy-file=/etc/kubernetes/abac-policy.json
```

⌘ ~~Not Preferable; large policy file; complex to handle;~~

々 **RBAC** [Role Based Access Control] (Authorization mechanism 3)

⌘ It consists of 4 main things:

Role, RoleBinding, ClusterRole, ClusterRoleBinding

[comes under '*rbac.authorization.k8s.io*' api group]

⌘ Role

⌘ **rules** list will be here.

⌘ Each rule contains: apiGroup (list), resources (list), verbs (list)

```
rules:
- apiGroups: ["" ] # "" indicates the core API group
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
```

⌘ Role is namespace scoped.

```
metadata:
  namespace: default
  name: pod-reader
```

⌘ If namespace field is not provided,
it'll be created on the namespace present in *current-context*
(or)

--namespace <namespace name> in this namespace

⌘ Within resource (ex: pod), if don't want to give access to all
resources (ex: all pods)

resourceNames is the key.

```
rules:
- apiGroups: ["" ]
  resources: ["pods"]
  verbs: ["get", "create", "update"]
  resourceNames: ["blue", "orange"]
```

[only pods with names "blue" & "orange" will be accessible]

ClusterRole

- Same as Role, but it is not namespace scoped.

It is **cluster-scoped**.

- No need to provide [namespace](#) field.

RoleBinding

- Maps subject to roles (Role, ClusterRole).

[subject: certificate subjects, CN: user, O: group ...etc]

- It is **namespace-scoped**.

```
subjects:
- kind: User
  name: dev-user
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: developer
  apiGroup: rbac.authorization.k8s.io
```

- * here **kind** is not same as top level kind i.e. *kind: Pods*
- * kind in roleRef: *Role* or *ClusterRole*
- * kind in subjects: *User*, *Group*, or *ServiceAccount*
- * apiGroup in *subjects* seems unnecessary (*User*, *Group*)
ServiceAccount is a k8s resource, present in core api root path ([apiGroup: ""](#)).
- * Internally ([kind + apiGroup](#)) matching is there, so apiGroup is needed.
- * Here also, **namespace** should be given;
if namespace given: *RoleBinding* object will be created in that.
If not, *RoleBinding* object will be created in *current-context* or --
namespace (if provided) namespace.
- * **Role provided in RoleBinding must be present in same namespace.**
If ClusterRole, then no worries.
- * **Only Resources in same namespace where RoleBinding exist will be authorized whether the role is Role or ClusterRole.**

ClusterRoleBinding

- It is cluster-scoped.
- It can only map to ClusterRole. ~~Cannot map Role.~~
- Resources through out the cluster will be authorized in here.

- ⌘ After creating binding (RoleBinding or ClusterRoleBinding), ~~you cannot~~ change roleRef in imperative way.
 - ⌘ Roles (Role, ClusterRole): which resources and actions can be accessed.
RoleBinding (RoleBinding, ClusterRoleBinding): maps subjects to roles
[subject: certificate subjects, CN: user, O: group ...etc]
- ⌘ Webhook (Authorization mechanism 4)
 - ⌘ Uses third-party REST service to get the authorization about a user or service account.
 - ⌘ Ex: Open Policy Client
- ⌘ **AlwaysAllow** and **AlwaysDeny** (Authorization mechanism 5 & 6)
 - ⌘ When no authorization mechanism provided in **--authorization-mode** *AlwaysAllow* is the default one.
 - ⌘ **AlwaysAllow** allows everyone to access everyone.
 - AlwaysDeny** restricts everyone's access to everything.
- ⌘ To see if a user is authorized (**auth can-i**):
kubectl auth can-i create deployments
yes or *no* (output)
To see if another user is authorized or not (**--as <user name>**)
kubectl auth can-i create deployments --as dev-user
yes or *no* (output)
- ⌘ Image Security
 - ⌘ Default docker registry: *docker hub*
 - ⌘ image: nginx
registry: docker.io
namespace (organization/user): library (official images present in this)
repository: nginx
tag: latest
 - ⌘ image: nginx ==> image: docker.io/library/nginx
 - ⌘ image: private-registry.io/apps/internal-app
[if you want to specify other registry or other namespace]
 - ⌘ In case of private registry, you need to authenticate in that.
 - ⌘ Create a **Secret** (Kubernetes Resource)
mention secret object's name in pod.spec.imagePullSecrets.name field.

```

❧ kubectl create secret docker-registry <secret name> \
--docker-server=<docker registry server> \
--docker-username=<docker registry username> \
--docker-password=<docker registry password> \
--docker-email=<docker registry email>

```

❧ `docker-registry =>` Create a secret for use with a Docker registry

```

spec:
  imagePullSecrets:
    - name: my-registry-secret-1
    - name: my-registry-secret-2

```

❧ `pod.spec.imagePullSecrets` is a list; it'll try one by one until it finds the required image.

[containers and imagePullSecrets: both are siblings]

❧ Create **secret** from yaml

```

apiVersion: v1
data:
  .dockerconfigjson: eyJhdXNldjI6eyJl
kind: Secret
metadata:
  name: my-secret
type: kubernetes.io/dockerconfigjson

```

[yaml file looks something like this]

❧ `secret.data..dockerconfigjson` (`\.` means to keep `.` as a string)

❧ That is **base64** encoded.

On decoding that:

```

{
  "auths": {
    "DOCKER_REGISTRY_SERVER": {
      "username": "DOCKER_USER",
      "password": "DOCKER_PASSWORD",
      "email": "DOCKER_EMAIL",
      "auth": "RE9DS0VSX1VTRVI6RE9DS0VSX1BBU1NXT1JE"
    }
  }
}

```

[something like

this will come]

❧ [official docs](#)

🔗 Security Context

❧ Docker containers default user: **root** [root's user ID: 0]

even *root* user doesn't have all capabilities. You need to specifically add.

❧ **docker run --user=**<user id> <image name> [user ID will be given]

docker run --cap-add <capability name> <image name> [add capability]

```

NET_ADMIN → control networking
SYS_ADMIN → system-level control
CHOWN → change file ownership
MAC_ADMIN → control security policies

```

- ⌘ In kubernetes, either you can define these in **pod level** or **container level**.

Pod level: [pod.spec.securityContext](#) [write the configuration here]

Container level: [pod.spec.containers\[*\].securityContext](#)

```

securityContext:
  runAsUser: 1000
  capabilities:
    add: ["MAC_ADMIN"]

```

🔗 NetworkPolicy ([apiVersion: networking.k8s.io/v1](#))

- ⌘ It is **namespace scoped**.
- ⌘ Default behaviour: any pod can communicate with any other pod.
Network Policies: restrict those behaviour.
- ⌘ Example: only backend should talk to database; not frontend;
- ⌘ Ingress: incoming traffic
egress: outgoing traffic
- ⌘ Traffic restriction will work only to the pods where NetworkPolicy is applied.
To link pods, **selector** and **labels** method only is used here as well.
- ⌘ [networkpolicy.spec.podSelector](#)
select the pod on which NetworkPolicy has to be applied i.e. on which ingress and egress traffic restriction will be applied.
- ⌘ [networkpolicy.spec.ingress](#)
rules for allowed ingress traffic to the pod. (target pod's selector, ports)
- ⌘ [networkpolicy.spec.egress](#)
rules for allowed egress traffic
- ⌘ [networkpolicy.spec.policyTypes](#)
it enforces the ingress and egress rules.
 - ⌘ If *policyTypes* field is not there, Kubernetes will add the values (Ingress, Egress) based on presence of [networkpolicy.spec.ingress](#) and [networkpolicy.spec.egress](#) fields.
If only ingress is present, it becomes Ingress; if only egress is present, it becomes Egress; if both are present, it becomes both

Ingress and Egress.

In the live configuration (**kubectl get networkpolicy <name> -o yaml**), the *policyTypes* field will always be present, either defined by the user or added by Kubernetes.

- In the live config, whatever values are there [Ingress or Egress or both] inside *networkpolicy.spec.policyTypes*, only those will be enforced.

If a direction [Ingress or Egress] is listed in *policyTypes* but no corresponding rules are defined, then that direction becomes **completely blocked (default deny)**.

For example, if *policyTypes* contains Ingress but no ingress rules are defined, then all incoming traffic is blocked.

```
apiVersion: networking.k8s.io/
kind: NetworkPolicy
metadata:
  name: test-network-policy
  namespace: default
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - ipBlock:
            cidr: 172.17.0.0/16
            except:
              - 172.17.1.0/24
        - namespaceSelector:
            matchLabels:
              project: myproject
        - podSelector:
            matchLabels:
              role: frontend
      ports:
        - protocol: TCP
          port: 6379
  egress:
    - to:
        - ipBlock:
            cidr: 10.0.0.0/24
      ports:
        - protocol: TCP
          port: 5978
```

- *networkpolicy.spec.ingress.from.namespaceSelector*

if provided, namespaces with those labels will be selected.

If not provided, pods within the same namespace (~~not all namespaces~~) will be allowed.

^

々 F

々 F

々 F

Linux Namespace

- Network Namespace is a separate network inside the host.
 - Host and Namespace doesn't know anything about each other.
- The namespace will have its own *network interfaces*, *arp table*, *routing table* (independent of Host).
- I've 2 vms: vm1, vm2
 - vm1: 10.10.10.10/24
 - vm2: 10.10.10.11/24
- vm1 and vm2 can ping each other because both belong to same LAN (10.10.10.x).

```
vagrant@vm1:~$ route
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
default 10.0.2.2 0.0.0.0 UG 100 0 0 enp0s3
10.0.2.0 0.0.0.0 255.255.255.0 U 100 0 0 enp0s3
10.0.2.2 0.0.0.0 255.255.255.255 UH 100 0 0 enp0s3
10.10.10.0 0.0.0.0 255.255.255.0 U 0 0 0 enp0s8
192.168.0.1 10.0.2.2 255.255.255.255 UGH 100 0 0 enp0s3
```

route table]

Gateway 0.0.0.0 means this subnet (10.10.10.0) Gateway/Router is not needed because both belong to same **subnet**.

10.10.10.0 is added on **enpos8** (means this interface is connected to private network i.e. vm to vm communication)

- ip add**
display all the network interfaces names like *lo* (loopback), *enpos3*, *enpos8* ..etc.
en: ethernet; **po**: PCI bus 0; **s3**: slot 3
enpos8: ethernet device on slot 8
in our VM (vm1, vm2), *lo enpos3 enpos8* will be there.
s3 is for vm-to-vm communication (inter LAN), **s8** is for internet access (NAT).

- To create a namespace

ip netns add <namespace name>

- We'll create 2 namespaces having name *red*, *blue* .

```
root@vm1:~# ip netns add red
root@vm1:~# ip netns add blue
```

♣ **ip netns list**

to list all the namespaces

```
root@vm1:~# ip netns list
blue
red
```

♪ To execute any command inside a namespace

ip netns exec <network namespace name> <command to execute>

♣ **ip netns exec red ip add**

```
root@vm1:~# ip netns exec red ip add
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

[only loopback is present]

[same for **blue** namespace as well]

♣ **ip netns exec red route**

```
root@vm1:~# ip netns exec red ip route
root@vm1:~#
```

[no route is present]

[same for **blue** namespace as well]

♪ Now we'll create a **veth pair** (just like a virtual cable) to connect the 2 namespaces.

ip link add <veth's one end name> **type veth peer name** <veth's another end name>

[when you create a **veth pair**, it creates 2 **veth** network interface in the host and connect them internally. From outside, it looks 2 ends of a virtual ethernet cable]

♣ **ip link add veth-red type veth peer name veth-blue**

```
valid_iface forever preferred_iface forever
4: veth-blue@veth-red: <BROADCAST,MULTICAST,M-DOWN> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether ba:86:30:88:3b:b8 brd ff:ff:ff:ff:ff:ff
5: veth-red@veth-blue: <BROADCAST,MULTICAST,M-DOWN> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether de:62:4d:8e:da:b9 brd ff:ff:ff:ff:ff:ff
```

[**ip add**]

[2 veth network interface; internally connected; both represents one end]

veth-blue@veth-red -> veth-blue interface is connected to veth-red

veth-red@veth-blue -> veth-red interface is connected to veth-blue

♣ If both the ends of a veth pair present in same place (ex: both in host), then it is displayed like

`end1-name@end2-name` (end1 to end2)

`end2-name@end1-name` (end2 to end1)

if both are present in different namespaces (host & namespace; or namespace1 & namespace2), then it'll be displayed like

`end1-name@if<end2's index at its namespace>`

`end2-name@if<end1's index at its namespace>`

々 To connect **veth** ends to namespaces

`ip link set <veth end name> netns <network namespace name>`

々 Connect *veth-red* end to the namespace **red**

`ip link set veth-red netns red`

⌘ Now if you see,

```
4: veth-blue@if5: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
   link/ether ba:86:30:88:3b:b8 brd ff:ff:ff:ff:ff:ff link-netns red
```

link-netns red it means other end is present in namespace red.

⌘ *veth-red@veth-blue* is gone

because *veth-red* is now connected to a namespace. Host linux (host) cannot show peer interface directly it moved to namespace.

⌘ *veth-blue@veth-red* got updated to *veth-blue@if5* because veth-red's index inside namespace **red** is **5**.

```
root@vm1:~# ip netns exec red ip add
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
5: veth-red@if4: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
   link/ether de:62:4d:8e:da:b9 brd ff:ff:ff:ff:ff:ff link-netnsid 0
```

[`ip netns exec red ip add`]

And also, here, *veth-red@if4* because *veth-blue* is at 4th index .

link-netnsid 0 means other end present in host.

々 Connect *veth-blue* to the namespace **blue**

`ip link set veth-blue netns blue`

⌘ Now, *veth-blue* will be gone from host because it belongs to namespace blue.

々 Network interfaces in both namespace now

⌘ Namespace **red**:

`ip netns exec red ip add`

```

root@vm1:~# ip netns exec red ip add
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
5: veth-red@if4: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
   link/ether de:62:4d:8e:da:b9 brd ff:ff:ff:ff:ff:ff link-netns blue

```

↪ **link-netns blue** : other end is in namespace blue

↪ Namespace **blue**:

ip netns exec blue ip add

```

root@vm1:~# ip netns exec blue ip add
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
4: veth-blue@if5: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
   link/ether ba:86:30:88:3b:b8 brd ff:ff:ff:ff:ff:ff link-netns red

```

↪ **link-netns red** : other end is in namespace red

↪ Now make the network interfaces in both namespaces UP.

ip netns exec <network namespace name> ip link set <interface name> up

↪ **ip netns exec red ip link set lo up**

ip netns exec red ip link set veth-red up

```

root@vm1:~# ip netns exec red ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
5: veth-red@if4: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state LOWERLAYERDOWN group default qlen 1000
   link/ether de:62:4d:8e:da:b9 brd ff:ff:ff:ff:ff:ff link-netns blue

```

LOWERLAYERDOWN: means it is up, but its other end (veth-blue) is down.

↪ **ip netns exec red ip link set lo up**

ip netns exec red ip link set veth-red up

```

root@vm1:~# ip netns exec blue ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
4: veth-blue@if5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
   link/ether ba:86:30:88:3b:b8 brd ff:ff:ff:ff:ff:ff link-netns red
   inet6 fe80::b886:30ff:fe88:3bb8/64 scope link
       valid_lft forever preferred_lft forever

```

↪ Now, both *veth-red* and *veth-blue* are UP.

↪ Assign IP to both the namespace

ip netns exec <namespace name> ip addr add <IP>/<subnet mask length>

dev <interface name>

- Assign in namespace **red**:

`ip netns exec red ip addr add 192.168.15.2/24 dev veth-red`

```
root@vm1:~# ip netns exec red ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN gr
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
5: veth-red@if4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqu
    link/ether de:62:4d:8e:da:b9 brd ff:ff:ff:ff:ff:ff link-netns blue
    inet 192.168.15.2/24 scope global veth-red
        valid_lft forever preferred_lft forever
    inet6 fe80::dc62:4dff:fe8e:dab9/64 scope link
        valid_lft forever preferred_lft forever
```

[IP is assigned]

- Assign in namespace **blue**:

`ip netns exec blue ip addr add 192.168.15.3/24 dev veth-blue`

```
root@vm1:~# ip netns exec blue ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN g
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
4: veth-blue@if5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc no
    link/ether ba:86:30:88:3b:b8 brd ff:ff:ff:ff:ff:ff link-netns red
    inet 192.168.15.3/24 scope global veth-blue
        valid_lft forever preferred_lft forever
    inet6 fe80::b886:30ff:fe88:3bb8/64 scope link
        valid_lft forever preferred_lft forever
```

- The IPs are 192.168.15.2/24 & 192.168.15.3/24,
so private LAN's network IP will be: **198.168.15.0**
& subnet mask will be: **255.255.255.0**

🔗 Ping the namespaces from each other. It'll work fine.

- From namespace **red**:

`ip netns exec red ping -c 4 192.168.15.3`

```
root@vm1:~# ip netns exec red ping -c 4 192.168.15.3
PING 192.168.15.3 (192.168.15.3) 56(84) bytes of data.
 64 bytes from 192.168.15.3: icmp_seq=1 ttl=64 time=0.041 ms
 64 bytes from 192.168.15.3: icmp_seq=2 ttl=64 time=0.135 ms
 64 bytes from 192.168.15.3: icmp_seq=3 ttl=64 time=0.060 ms
 64 bytes from 192.168.15.3: icmp_seq=4 ttl=64 time=0.060 ms

--- 192.168.15.3 ping statistics ---
 4 packets transmitted, 4 received, 0% packet loss, time 3071ms
 rtt min/avg/max/mdev = 0.041/0.074/0.135/0.036 ms
```


- From namespace **blue**:

`ip netns exec blue ping -c 4 192.168.15.2`

```
root@vm1:~# ip netns exec blue ping -c 4 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
64 bytes from 192.168.15.2: icmp_seq=1 ttl=64 time=0.210 ms
64 bytes from 192.168.15.2: icmp_seq=2 ttl=64 time=0.091 ms
64 bytes from 192.168.15.2: icmp_seq=3 ttl=64 time=0.091 ms
64 bytes from 192.168.15.2: icmp_seq=4 ttl=64 time=0.057 ms

--- 192.168.15.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 0.057/0.112/0.210/0.058 ms
```

- Check **arp** table in both namespace, it should be updated.

- In namespace **red**:

```
root@vm1:~# ip netns exec red arp
Address          Hwtype  Hwaddress      Flags Mask      Iface
192.168.15.3      ether   ba:86:30:88:3b:b8  C              veth-red
```

- In namespace **blue**:

```
root@vm1:~# ip netns exec blue arp
Address          Hwtype  Hwaddress      Flags Mask      Iface
192.168.15.2      ether   de:62:4d:8e:da:b9  C              veth-blue
```

- But in Host's arp table, nothing related to namespaces will be there.

Because both are separate.

```
root@vm1:~# arp
Address          Hwtype  Hwaddress      Flags Mask      Iface
10.0.2.2          ether   52:54:00:12:35:00  C          enp0s3
10.10.10.11        ether   08:00:27:ad:99:10  C          enp0s8
```

- Try to ping the namespaces from Host.

`ping -c 4 <namespace network interface IP>`

- Ping to namespace **red**:

`ping -c 4 192.168.15.2`

```
root@vm1:~# ping -c 4 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
^Z
[4]+  Stopped                  ping -c 4 192.168.15.2
```

FAILED

- Ping to namespace **red**

`ping -c 4 192.168.15.3`

```
root@vm1:~# ping -c 4 192.168.15.3
PING 192.168.15.3 (192.168.15.3) 56(84) bytes of data.
^Z
[5]+  Stopped                  ping -c 4 192.168.15.3
```

FAILED

- ⌘ It is failing because there is no connection between host and namespaces.

Create a veth pair and connect that to host and namespace, then only you can ping.

[however, both end present in the host initially, just connect one to a namespace]

- ✧ To test connection between host and a namespace (let **red**), create a veth pair and connect one end to the namespace.

Another end will be by default connected to host.

- ⌘ `ip link add veth2-red type veth peer name veth2-red-host` [create veth pair]
- ⌘ `ip link set veth2-red netns red` [connect veth2-red end to namespace red]
- ⌘ `ip netns exec red ip link set veth2-red up` [make the link up in namespace]
- ⌘ `ip link set veth2-red-host up` [make the link up in host]
- ⌘ `netns exec red ip addr add 192.168.25.2/24 dev veth2-red` [assign ip in namespace]
- ⌘ `ip addr add 192.168.25.3/24 dev veth2-red-host` [assign ip in host]
- ⌘ `ping -c 4 192.168.25.2` (ping from host to namespace)

```
root@vm1:~# ping -c 4 192.168.25.2
PING 192.168.25.2 (192.168.25.2) 56(84) bytes of data.
64 bytes from 192.168.25.2: icmp_seq=1 ttl=64 time=2.54 ms
64 bytes from 192.168.25.2: icmp_seq=2 ttl=64 time=0.058 ms
64 bytes from 192.168.25.2: icmp_seq=3 ttl=64 time=0.069 ms
64 bytes from 192.168.25.2: icmp_seq=4 ttl=64 time=0.086 ms

--- 192.168.25.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3041ms
rtt min/avg/max/mdev = 0.058/0.688/2.542/1.070 ms
```

- ⌘ `ip netns exec red ping -c 4 192.168.25.3` (ping from namespace to host)

```
root@vm1:~# ip netns exec red ping -c 4 192.168.25.3
PING 192.168.25.3 (192.168.25.3) 56(84) bytes of data.
64 bytes from 192.168.25.3: icmp_seq=1 ttl=64 time=0.327 ms
64 bytes from 192.168.25.3: icmp_seq=2 ttl=64 time=0.062 ms
64 bytes from 192.168.25.3: icmp_seq=3 ttl=64 time=0.052 ms
64 bytes from 192.168.25.3: icmp_seq=4 ttl=64 time=0.307 ms

--- 192.168.25.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 0.052/0.187/0.327/0.130 ms
```

- ⌘ Here we did the same thing, before connected *veth pair* from namespace to namespace; not connected *veth pair* from host to namespace.
But, we had pinged to **192.168.25.2** (not **15.2** which is previous veth's subnet)
- ⌘ You can add gateway in host
ip route add <network-ip> via <gateway-ip>
ip route add 192.168.15.0 via 192.168.25.2
but need to enable packet forwarding inside namespace **red**. Because linux doesn't forward packets by default.
- ⌘ But, one issue will be there,
we configured 192.168.15.0 via 192.168.25.2
but the namespace **blue** also have this subnet, for that also it'll send to **red** namespace because gateway is 192.168.25.2
- ⌘ We'll do this in broader picture; not namespace level;
- 々 We had 2 namespaces so one veth and directly connect both.
In case of multiple namespaces, need to create a bridge.
[inside linux, **virtual switch** can be created]
- 々 Delete all the veth pairs.
ip link delete <any one end name>
[it'll delete the other name by default]
 - ⌘ **ip netns exec red ip link delete veth-red**
 - ⌘ **ip link delete veth2-red-host**
- 々 Create a bridge
ip link add <bridge interface name> type bridge
⌘ **ip link add v-net-0 type bridge**

```
8: v-net-0: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether fe:65:b1:8d:47:f6 brd ff:ff:ff:ff:ff:ff
```

 ⌘ [ip add]
its coming as a normal network interface only.
- ⌘ Linux bridge is nothing but a network interface like others.
- ⌘ The bridge belongs to the namespace where it is created.
If created in host, it belong to the host.
If created in a namespace, it belong to that namespace.

々 Make the bridge network interface UP.

`ip link set v-net-0 up`

々 Now create a **veth pair**, connect one end to the namespace **red** and another to **bridge**.

♣ `ip link add veth-red type veth peer name veth-red-br` [create veth pair]

♣ `ip link set veth-red netns red` [connect one end to namespace *red*]

♣ Now in host [`ip add`]

```
8: v-net-0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default qlen 1000
   link/ether fe:65:b1:8d:47:f6 brd ff:ff:ff:ff:ff:ff
9: veth-red-br@if10: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
   link/ether be:57:96:43:87:7c brd ff:ff:ff:ff:ff:ff link-netns red
```

♣ `ip link set veth-red-br master v-net-0` [connect another end to bridge]

[for master node's interface, use **master** <host's interface name>]

♣ `ip netns exec red ip link set veth-red up` [make namespace end up]

♣ `ip link set veth-red-br up` [make bridge end up]

々 Do the same for **blue** namespace.

♣ `ip link add veth-blue type veth peer name veth-blue-br` [create veth pair]

♣ `ip link set veth-blue netns blue` [connect one end to namespace *red*]

♣ Now in host [`ip add`]

```
11: veth-blue-br@if12: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
   link/ether 06:8d:2b:07:c6:00 brd ff:ff:ff:ff:ff:ff link-netns blue
```

♣ `ip link set veth-blue-br master v-net-0` [connect another end to bridge]

♣ `ip netns exec blue ip link set veth-blue up` [make namespace end up]

♣ `ip link set veth-blue-br up` [make bridge end up]

々 Now assign IPs to both red and blue namespace

♣ `ip netns exec red ip addr add 192.168.15.2/24 dev veth-red` [assign IP to red namespace]

♣ `ip netns exec blue ip addr add 192.168.15.3/24 dev veth-blue` [assign IP to blue namespace]

々 Now, ping from one namespace to another will be successful.

- ⌘ Ping red to blue

`ip netns exec red ping -c 4 192.168.15.3`

```
root@vm1:~# ip netns exec red ping -c 4 192.168.15.3
PING 192.168.15.3 (192.168.15.3) 56(84) bytes of data.
64 bytes from 192.168.15.3: icmp_seq=1 ttl=64 time=3.58 ms
64 bytes from 192.168.15.3: icmp_seq=2 ttl=64 time=0.103 ms
64 bytes from 192.168.15.3: icmp_seq=3 ttl=64 time=0.147 ms
64 bytes from 192.168.15.3: icmp_seq=4 ttl=64 time=0.080 ms

--- 192.168.15.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3044ms
rtt min/avg/max/mdev = 0.080/0.977/3.580/1.502 ms
```

- ⌘ Ping from blue to red

`ip netns exec red ping -c 4 192.168.15.3`

```
root@vm1:~# ip netns exec blue ping -c 4 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
64 bytes from 192.168.15.2: icmp_seq=1 ttl=64 time=0.501 ms
64 bytes from 192.168.15.2: icmp_seq=2 ttl=64 time=0.060 ms
64 bytes from 192.168.15.2: icmp_seq=3 ttl=64 time=0.107 ms
64 bytes from 192.168.15.2: icmp_seq=4 ttl=64 time=0.068 ms

--- 192.168.15.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3046ms
rtt min/avg/max/mdev = 0.060/0.184/0.501/0.183 ms
```

♪ But, Host cannot ping either of these namespace

- ⌘ Host to namespace red

```
root@vm1:~# ping -c 4 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
^Z
[10]+  Stopped                  ping -c 4 192.168.15.2
```

- ⌘ Host to namespace blue

```
root@vm1:~# ping -c 4 192.168.15.3
PING 192.168.15.3 (192.168.15.3) 56(84) bytes of data.
^Z
[11]+  Stopped                  ping -c 4 192.168.15.3
```

♪ Add IP address to the bridge network interface (v-net-o)

`ip addr add 192.168.15.5/24 dev v-net-o`

- ⌘ Because till now host doesn't know any subnet of 102.168.15.x

♪ Now try to ping again.

- ⌘ Not it'll be successful

because, host has now a interface having the same subnet as that of the namespaces.

- Host to namespace red

```
root@vm1:~# ping -c 4 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
64 bytes from 192.168.15.2: icmp_seq=1 ttl=64 time=3.45 ms
64 bytes from 192.168.15.2: icmp_seq=2 ttl=64 time=0.373 ms
64 bytes from 192.168.15.2: icmp_seq=3 ttl=64 time=0.111 ms
64 bytes from 192.168.15.2: icmp_seq=4 ttl=64 time=0.057 ms

--- 192.168.15.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3059ms
rtt min/avg/max/mdev = 0.057/0.998/3.452/1.421 ms
```

- Host to namespace blue

```
root@vm1:~# ping -c 4 192.168.15.3
PING 192.168.15.3 (192.168.15.3) 56(84) bytes of data.
64 bytes from 192.168.15.3: icmp_seq=1 ttl=64 time=2.51 ms
64 bytes from 192.168.15.3: icmp_seq=2 ttl=64 time=0.248 ms
64 bytes from 192.168.15.3: icmp_seq=3 ttl=64 time=0.117 ms
64 bytes from 192.168.15.3: icmp_seq=4 ttl=64 time=0.090 ms

--- 192.168.15.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 0.090/0.740/2.506/1.021 ms
```

- ✧ Ping from namespace to host should also work.

But to **102.168.15.5** (bridge network interface IP) only.

It'll not work for *10.10.10.10* (host (vm1) IP)

- Namespace red to host

```
root@vm1:~# ip netns exec red ping -c 4 192.168.15.5
PING 192.168.15.5 (192.168.15.5) 56(84) bytes of data.
64 bytes from 192.168.15.5: icmp_seq=1 ttl=64 time=0.894 ms
64 bytes from 192.168.15.5: icmp_seq=2 ttl=64 time=0.066 ms
64 bytes from 192.168.15.5: icmp_seq=3 ttl=64 time=0.083 ms
64 bytes from 192.168.15.5: icmp_seq=4 ttl=64 time=0.059 ms

--- 192.168.15.5 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3043ms
rtt min/avg/max/mdev = 0.059/0.275/0.894/0.357 ms
```

- Namespace blue to host.

```
root@vm1:~# ip netns exec blue ping -c 4 192.168.15.5
PING 192.168.15.5 (192.168.15.5) 56(84) bytes of data.
64 bytes from 192.168.15.5: icmp_seq=1 ttl=64 time=1.58 ms
64 bytes from 192.168.15.5: icmp_seq=2 ttl=64 time=0.080 ms
64 bytes from 192.168.15.5: icmp_seq=3 ttl=64 time=0.066 ms
64 bytes from 192.168.15.5: icmp_seq=4 ttl=64 time=0.066 ms

--- 192.168.15.5 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3038ms
rtt min/avg/max/mdev = 0.066/0.448/1.581/0.654 ms
```

- ✧ To *10.10.10.10* will not work

```
root@vm1:~# ip netns exec red ping -c 4 10.10.10.10
ping: connect: Network is unreachable
```

```
root@vm1:~# ip netns exec blue ping -c 4 10.10.10.10
ping: connect: Network is unreachable
```

['Network is unreachable' comes when **gateway is not there**]

♪ Add gateway in the namespaces

network subnet: 10.10.10.0

gateway: 192.168.15.5 [bridge network interface's IP]

[means to communicate with 10.10.10.0, route to 192.168.15.5 (bridge's IP)]

↪ `ip netns exec red ip route add 10.10.10.0/24 via 192.168.15.5` [in namespace red]

↪ `ip netns exec blue ip route add 10.10.10.0/24 via 192.168.15.5` [in namespace blue]

[always add the subnet /24, otherwise it'll take /32 by default]

♪ Now try to ping host (10.10.10.10) from namespaces

↪ Namespace red to host

```
root@vm1:~# ip netns exec red ping -c 4 10.10.10.10
PING 10.10.10.10 (10.10.10.10) 56(84) bytes of data.
64 bytes from 10.10.10.10: icmp_seq=1 ttl=64 time=3.43 ms
64 bytes from 10.10.10.10: icmp_seq=2 ttl=64 time=0.074 ms
64 bytes from 10.10.10.10: icmp_seq=3 ttl=64 time=0.085 ms
64 bytes from 10.10.10.10: icmp_seq=4 ttl=64 time=0.122 ms

--- 10.10.10.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3037ms
rtt min/avg/max/mdev = 0.074/0.928/3.431/1.445 ms
```

↪ Namespace blue to host

```
root@vm1:~# ip netns exec blue ping -c 4 10.10.10.10
PING 10.10.10.10 (10.10.10.10) 56(84) bytes of data.
64 bytes from 10.10.10.10: icmp_seq=1 ttl=64 time=0.917 ms
64 bytes from 10.10.10.10: icmp_seq=2 ttl=64 time=0.097 ms
64 bytes from 10.10.10.10: icmp_seq=3 ttl=64 time=0.063 ms
64 bytes from 10.10.10.10: icmp_seq=4 ttl=64 time=0.075 ms

--- 10.10.10.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3036ms
rtt min/avg/max/mdev = 0.063/0.288/0.917/0.363 ms
```

♪ Now try to ping **vm2** (10.10.10.11) from the namespaces.

It'll ping (because gateway is there) but response will not come.

↪ Namespace red to vm2

```
root@vm1:~# ip netns exec red ping -c 4 10.10.10.11
PING 10.10.10.11 (10.10.10.11) 56(84) bytes of data.
^Z
[14]+  Stopped                  ip netns exec red ping -c 4 10.10.10.11
```

- Namespace blue to vm2

```
root@vm1:~# ip netns exec blue ping -c 4 10.10.10.11
PING 10.10.10.11 (10.10.10.11) 56(84) bytes of data.
^Z
[15]+  Stopped                  ip netns exec blue ping -c 4 10.10.10.11
```

- It is because, packet is going to vm2 (10.10.10.11), but it doesn't know about **192.168.15.x** because it belongs to private network of **vm1** (host).
- Either add a gateway in **vm2** or enable **NAT** in vm1.

Option-1: adding gateway in **vm2**

network: 192.168.15.0

gateway: 10.10.10.10 (vm1 / host)

[means to communicate with 192.168.15.0 subnet, route to 10.10.10.10]

- ip route add 192.168.15.0/24 via 10.10.10.10 [vm2]**

```
root@vm2:~# ip route
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.0.2.2 dev enp0s3 proto dhcp scope link src 10.0.2.15 metric 100
10.10.10.0/24 dev enp0s8 proto kernel scope link src 10.10.10.11
192.168.0.1 via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
192.168.15.0/24 via 10.10.10.10 dev enp0s8
```

- [vm2]

[ip route]

- Even now ping from namespace to vm2 or vice-versa will not work.

Because, **IP forwarding is disable in vm1** (host)

cat /proc/sys/net/ipv4/ip_forward

0: disabled

1: enabled

- vm1 OS** is acting as the **gateway** and **router** for different networks. 10.10.10.x network & 192.168.15.x network.
- A router's main job is to receive packet from a network, and forward that to another network.
Here, **vm1 OS** is acting as a router, receiving from one network and forwarding to another. So, packet forwarding needs to be enabled.
- Enable ip forwarding
echo 1 > /proc/sys/net/ipv4/ip_forward

- Now ping will work

```
root@vm2:~# ping 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
64 bytes from 192.168.15.2: icmp_seq=1 ttl=63 time=4.68 ms
64 bytes from 192.168.15.2: icmp_seq=2 ttl=63 time=1.55 ms
64 bytes from 192.168.15.2: icmp_seq=3 ttl=63 time=1.31 ms
^Z
[8]+  Stopped                  ping 192.168.15.2
```

[vm2 to

namespace]

```
root@vm1:~# ip netns exec red ping 10.10.10.11
PING 10.10.10.11 (10.10.10.11) 56(84) bytes of data.
64 bytes from 10.10.10.11: icmp_seq=1 ttl=63 time=1.90 ms
64 bytes from 10.10.10.11: icmp_seq=2 ttl=63 time=0.902 ms
64 bytes from 10.10.10.11: icmp_seq=3 ttl=63 time=2.23 ms
64 bytes from 10.10.10.11: icmp_seq=4 ttl=63 time=1.20 ms
^Z
[20]+  Stopped                  ip netns exec red ping 10.10.10.11
```

[namespace t

vm2]

- In this case, **vm2** can directly communicate to **namespace** (of **vm1**).

Option-2: using NAT

- Previous option is not preferable because if multiple systems will be there, in each system we need to set route gateway.
- If we enable NAT in **vm1**, it'll update the source IP before the packet exits.

- Packet before NAT:

src IP: 192.168.15.2 (red namespace)

dest IP: 10.10.10.11 (vm2)

- Packet after NAT:

src IP: 10.10.10.10 (host/vm1 IP)

dest IP: 10.10.10.11 (vm2) --> it doesn't change.

- Now the packet can go to **vm2** easily, and vm2 as well send back the response.

src IP: 10.10.10.11

dest IP: 10.10.10.10 (because this was the source IP received at vm2)

- vm1 (host) will receive the packet. And from the NAT table it knows that this packet belong to 192.168.15.2 (red namespace).

Now it updates the packet's dest IP

src IP: 10.10.10.11 (vm2, doesn't change)

dest IP: 192.158.15.2 (red namespace)

- ⌘ To setup this, First delete the created configs (vm2 route gateway).
Don't disable IP packet forwarding (/proc/sys/net/ipv4/ip_forward) because NAT requires this.

- ⌘ **iptables -t nat -A POSTROUTING -o <network interface name> -j MASQUERADE**

-t: use NAT table

POSTROUTING: modify packet before leaving

<network interface name>: network interface connected to vm2

```
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu
    link/ether 08:00:27:c3:54:e8 brd ff:ff:ff:ff:
    inet 10.10.10.10/24 brd 10.10.10.255 scope gl
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fec3:54e8/64 scope link
        valid_lft forever preferred_lft forever
```

[in my case enpos8]

MASQUERADE: replace source IP with host/router IP

[for incoming traffic, destination ip will be updated (host IP to namespace's IP). it looks like DNAT but it is not. Kernel automatically reverse NAT mapping because it remembers previously source IP was changed]

- ⌘ **iptables -t nat -A POSTROUTING -o enpos8 -j MASQUERADE**

- ⌘ Now, from namespace to vm2 can be easily accessible.

```
root@vm1:~# ip netns exec red ping -c 4 10.10.10.11
PING 10.10.10.11 (10.10.10.11) 56(84) bytes of data.
64 bytes from 10.10.10.11: icmp_seq=1 ttl=63 time=3.74 ms
64 bytes from 10.10.10.11: icmp_seq=2 ttl=63 time=1.83 ms
64 bytes from 10.10.10.11: icmp_seq=3 ttl=63 time=1.90 ms
64 bytes from 10.10.10.11: icmp_seq=4 ttl=63 time=1.03 ms

--- 10.10.10.11 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3009ms
rtt min/avg/max/mdev = 1.027/2.123/3.736/0.992 ms
```

[red to vm2]

```
root@vm1:~# ip netns exec blue ping -c 4 10.10.10.11
PING 10.10.10.11 (10.10.10.11) 56(84) bytes of data.
64 bytes from 10.10.10.11: icmp_seq=1 ttl=63 time=3.14 ms
64 bytes from 10.10.10.11: icmp_seq=2 ttl=63 time=1.29 ms
64 bytes from 10.10.10.11: icmp_seq=3 ttl=63 time=1.49 ms
64 bytes from 10.10.10.11: icmp_seq=4 ttl=63 time=0.906 ms

--- 10.10.10.11 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 0.906/1.707/3.140/0.853 ms
```

[blue to vm2]

- ⌘ But, vm2 to namespace will not be accessible.

Namespace to vm2:

when vm2 receives the packet, *src ip: vm1's ip*, *dest ip: vm2's ip*

so, vm2 also sends the packet, *src ip: vm2's ip*, *dest ip: vm1's ip*
at the end, vm1 forward the packet to namespace.

[in this scenario, vm2 is sending packet to vm1 only, not to namespace]

- ⌘ But, directly if you try to access namespace from vm2 it'll not work because vm2 doesn't know about namespace.

If you want this only, then **route** needs to be added in **vm2** .

- ⌘ **Port forwarding** can be used in this case.

Vm2 will send packet to vm1 at a particular port. Vm1 will forward the packet to namespace.

It is **DNAT**.

- ⌘ **iptables -t nat -A PREROUTING -p tcp --dport 8080 -j DNAT --to-destination 192.168.15.2:80**

Packets arriving on port 8080 should be redirected to namespace 192.168.15.2:80

- ⌘ I ran a simple nodejs in namespace *red* on port 80,

curl <http://10.10.10.10:8080> [from vm2]

```
root@vm2:~# curl http://10.10.10.10:8080
Hello from Node.js server on port 80
```

々 All commands used

- ⌘ Show all network interfaces
`ip add`
- ⌘ Create a network namespace
`ip netns add <namespace-name>`
`ip netns add red`
- ⌘ List all namespaces
`ip netns list`
- ⌘ Execute command inside namespace
`ip netns exec <namespace-name> <command>`
`ip netns exec red ip add`
- ⌘ Show routing table inside namespace
`ip netns exec <namespace-name> route`
`ip netns exec red route`
- ⌘ Create veth pair
- ⌘ `ip link add <veth1> type veth peer name <veth2>`

- ♣ Example
- ♣ `ip link add veth-red type veth peer name veth-blue`
- ♣ Connect veth to namespace
- ♣ `ip link set <veth-name> netns <namespace-name>`
- ♣ Example
- ♣ `ip link set veth-red netns red`
- ♣ Bring interface up inside namespace
- ♣ `ip netns exec <namespace-name> ip link set <interface-name> up`
- ♣ Examples
- ♣ `ip netns exec red ip link set lo up`
- ♣ `ip netns exec red ip link set veth-red up`
- ♣ Assign IP inside namespace
- ♣ `ip netns exec <namespace-name> ip addr add <ip>/<mask> dev <interface-name>`
- ♣ Examples
- ♣ `ip netns exec red ip addr add 192.168.15.2/24 dev veth-red`
- ♣ `ip netns exec blue ip addr add 192.168.15.3/24 dev veth-blue`
- ♣ Ping from namespace
- ♣ `ip netns exec <namespace-name> ping -c 4 <ip>`
- ♣ Examples
- ♣ `ip netns exec red ping -c 4 192.168.15.3`
- ♣ `ip netns exec blue ping -c 4 192.168.15.2`
- ♣ Ping from host
- ♣ `ping -c 4 <ip>`
- ♣ Examples
- ♣ `ping -c 4 192.168.15.2`
- ♣ `ping -c 4 192.168.15.3`
- ♣ Add route in host
- ♣ `ip route add <network-ip> via <gateway-ip>`
- ♣ Example
- ♣ `ip route add 192.168.15.0 via 192.168.25.2`
- ♣ Delete interface
- ♣ `ip link delete <interface-name>`
- ♣ Examples

- ♣ `ip netns exec red ip link delete veth-red`
- ♣ `ip link delete veth2-red-host`
- ♣ Create bridge
- ♣ `ip link add <bridge-name> type bridge`
- ♣ Example
- ♣ `ip link add v-net-o type bridge`
- ♣ Bring bridge up
- ♣ `ip link set <bridge-name> up`
- ♣ Example
- ♣ `ip link set v-net-o up`
- ♣ Connect interface to bridge
- ♣ `ip link set <interface-name> master <bridge-name>`
- ♣ Examples
- ♣ `ip link set veth-red-br master v-net-o`
- ♣ `ip link set veth-blue-br master v-net-o`
- ♣ Assign IP to bridge
- ♣ `ip addr add <ip>/<mask> dev <bridge-name>`
- ♣ Example
- ♣ `ip addr add 192.168.15.5/24 dev v-net-o`
- ♣ Add route inside namespace
- ♣ `ip netns exec <namespace-name> ip route add <network>/<mask> via <gateway-ip>`
- ♣ Examples
- ♣ `ip netns exec red ip route add 10.10.10.0/24 via 192.168.15.5`
- ♣ `ip netns exec blue ip route add 10.10.10.0/24 via 192.168.15.5`
- ♣ Check IP forwarding
- ♣ `cat /proc/sys/net/ipv4/ip_forward`
- ♣ Enable IP forwarding
- ♣ `echo 1 > /proc/sys/net/ipv4/ip_forward`
- ♣ Enable NAT
- ♣ `iptables -t nat -A POSTROUTING -o <interface-name> -j MASQUERADE`
- ♣ Example
- ♣ `iptables -t nat -A POSTROUTING -o enpos8 -j MASQUERADE`
- ♣ Add DNAT port forwarding

⌘ iptables -t nat -A PREROUTING -p tcp --dport <host-port> -j DNAT --to-destination <ip>:<port>

⌘ Example

⌘ iptables -t nat -A PREROUTING -p tcp --dport 8080 -j DNAT --to-destination 192.168.15.2:80

⌘ Access forwarded application

⌘ curl http://<host-ip>:<port>

⌘ Example

⌘ curl http://10.10.10.10:8080

々 f

DNS (Domain Name Server)

♪ Let say, you are having 2 servers

vm1: 10.10.10.10

vm2: 10.10.10.11

[vm1 and vm2 are the host names of the respective servers]

♣ vm1 and vm2 can ping each other using their IP addresses.

ping 10.10.10.11 [from vm1]

ping 10.10.10.10 [from vm2]

♣ **/etc/hosts** file can be updated to get rid of writing IP every time.

```
10.10.10.10 vm1
10.10.10.11 vm2
```

[like this]

♣ Now, both can ping like this

ping vm2 [from vm1]

ping vm1 [from vm2]

♣ Let say I created one more entry

```
10.10.10.11 vm2
10.10.10.11 my-vm2
```

now I have 2 different names for the same IP (vm2), if I ping now

ping my-vm2

it'll work. It doesn't matter what is the actual hostname of the server having that IP.

Whatever you write inside the **/etc/hosts** file, that can only be used to communicate with the respective server.

♣ This process is called **name resolution**.

♪ If you have multiple servers, you need to write each server's IP with a name (hostname is preferred; but is not necessary).

So, A central server is configured which will keep all these IP to name mappings.

Now only one server needs to be updated instead of updating each systems. This is **DNS** server.

♪ You can maintain both **/etc/hosts** (for name resolution) and **/etc/resolve.conf** simultaneously.

♣ For this you can configure by yourself, what to see first.

- ♣ By default if you give any name (google.com for example), it'll first check in the **/etc/hosts** file, if not found then go to DNS server(s) and check.

- ♣ This behaviour can be configured in the file **/etc/nsswitch.conf**

```
hosts:          files dns
```

[this is one of the entries]

- ♣ **files** means check **/etc/hosts** file
- ♣ **dns** means check DNS servers mentioned inside **/etc/resolve.conf**
- ♣ If the order was like

hosts: dns files

then, first DNS then files.

- ♣ But it is not limited to *files* and *dns* only.

```
passwd:         files systemd
group:          files systemd
shadow:        files
gshadow:        files

hosts:          files dns
networks:       files

protocols:      db files
services:       db files
ethers:         db files
rpc:            db files
```

[can be **db**, **systemd** ..etc]

- ♪ To use a DNS server, you need to update **/etc/resolve.conf** file.

nameserver <IP of DNS server> [you can keep as many dns server you want]

```
nameserver 8.8.8.8
nameserver 127.0.0.53
```

[8.8.8.8 is Google's DNS server's IP]

- ♣ Now if you execute

ping google.com

or

ping facebook.com

it'll work.

- ♣ We mentioned Google's DNS server but how *facebook.com* worked?

Recursive DNS

- ♪ Flow of Domain resolving

- ♣ Typically in production the domain is stored in a provider like *Route53*, *GoDaddy* ..etc.

- ✧ These contains all the entries, for example
 - mycompany.com* [main domain, com: TLD (top level domain)]
 - prod.myapp.mycompany.com*
 - test.myapp.mycompany.com*
 - prod.otherapp.mycompany.com*
 - api.v1.myapp.mycompany.com*
 - www.mycompany.com* ... etc

here, whatever comes before *mycompany.com* (main domain), all are called **sub-domains**, there is no limit of the number of sub domains.

- ⌘ Application initiates request
 - Example: ping *page.alok.facebook.com*
- ⌘ System checks nsswitch.conf
 - hosts: files dns
 - ✧ first check */etc/hosts*
 - ✧ if not found, use *DNS*
- ⌘ System reads */etc/resolv.conf*
 - nameserver 127.0.0.53 [*local resolver*]
- ⌘ Local resolver (systemd-resolved) receives query
 - ✧ checks local cache
 - ✧ if not found, forwards request
- ⌘ Local resolver forwards query to router DNS
 - Example: 192.168.0.1
 - ✧ acts as DNS forwarder
- ⌘ Router DNS checks its cache
 - ✧ if found, return result
 - ✧ if not, forwards request to *Recursive DNS*
- ⌘ Router forwards query to recursive DNS server
 - Example: 8.8.8.8 (Google Public DNS)
 - ✧ this is the actual recursive resolver
- ⌘ Recursive DNS checks its cache
 - ✧ if found, return result
 - ✧ if not, starts full DNS resolution
- ⌘ Recursive DNS queries Root (🌐) servers

- ↳ asks: who handles .com
- ↳ Root server responds with TLD servers for .com
- ↳ Recursive DNS queries .com TLD server
 - ↳ asks: who handles facebook.com
- ↳ TLD server responds with authoritative nameservers for facebook.com
- ↳ Recursive DNS queries authoritative nameserver
 - ↳ asks: what is IP of page.alok.facebook.com
- ↳ Authoritative server checks zone records
 - ↳ finds matching record (or wildcard)
 - ↳ returns IP address
- ↳ Recursive DNS caches the result
- ↳ Recursive DNS returns IP to router (192.168.0.1)
- ↳ Router returns IP to local resolver (127.0.0.53)
- ↳ Local resolver caches the result
- ↳ Local resolver returns IP to application
- ↳ Application uses IP to communicate with server
- Example: ping / curl / browser connects to that IP
- ↳ Local Resolver (systemd-resolved) [↳ machine-level helper]
 - ↳ Runs on your machine (127.0.0.53)
 - ↳ Type: stub resolver
 - ↳ Receives DNS queries from applications
 - ↳ Checks local cache
 - ↳ Forwards queries to upstream DNS
 - ↳ Does NOT perform full DNS recursion (no root → TLD → authoritative)
- ↳ Router DNS (192.168.0.1) [↳ network-level forwarder]
 - ↳ Runs on your network router
 - ↳ Type: DNS forwarder (sometimes basic resolver)
 - ↳ Receives DNS queries from devices in network
 - ↳ Checks its cache
 - ↳ Forwards queries to ISP DNS or public DNS (e.g., 8.8.8.8)
 - ↳ Usually does NOT perform full recursion (depends on router)
- ↳ Both: Cache DNS results & Forward queries
- ↳ F

々 F

々 F

々 F

々 F

々 F

々 F

々 F

々